

Java – Beautifully Broken

Final, shallow and honest: what a record really promises

Issue #4

Sun, 04 Oct 2026

🕒 ~18 min read

Level 3/5

⚡ The short version

1. **A record is a promise, not a shortcut.** "This class is its data": final fields, generated `equals` / `hashCode`, and one canonical constructor where you enforce every invariant. Its immutability is **shallow**, though, so lists and arrays need care.
2. **Spring now releases everything on one "Patch Thursday" a month.** The whole portfolio ships on a single day. The first patch train under the new model is due **22 October**, two days after Oracle's CPU.
3. **Patch Jackson.** CVE-2026-68497 (CVSS 7.5) lets a tiny JSON request burn CPU through `Duration` / `XMLGregorianCalendar` fields. Fixed in **2.18.10**, **2.21.6**, **2.22.2**, **3.1.6** and **3.2.2**.

Version watch

25

current **LTS** (Sep 2025),
free Oracle updates until
Sep 2028

27

latest JDK, **GA September 2026** (non-LTS)

4.1.1

latest **Spring Boot** GA (20 Aug 2026); 4.2.0-M2 in milestones

29

next planned **LTS**,
September 2027

Sources: Oracle Java SE support roadmap, spring.io releases and endoflife.date (checked 4 Oct).

PART 1 Learn



Records: data carriers done right

CATCH UP

~10 MIN

LEVEL 3

Records arrived as a final feature in Java 16, so if your Java stopped at 8 or 11, you've been writing them by hand (or with Lombok) ever since. Yesterday's issue showed the payoff: a record key gets `equals` and `hashCode` right for free. Today is about the rest: **how to put validation in a record, what "immutable" really means for one, and where a record is the wrong tool.**

The design goal in JEP 395 isn't "less boilerplate". It's "**modeling data as data**": a record is a *transparent* carrier for immutable data, and the class declaration is the whole story. (JEP 395) Keep that idea in mind. Every rule below follows from it.

1 · WHAT ONE LINE BUYS YOU

A value type: then vs now

Java 8: hand-written

```
public final class Range {
    private final int lo, hi;
    public Range(int lo, int hi) {
        this.lo = lo; this.hi = hi;
    }
    public int getLo() { return lo; }
    public int getHi() { return hi; }
    @Override public boolean equals(..)
    @Override public int hashCode()
    @Override public String toString()
    // ~40 lines once written out,
    // and one more field means
    // editing five places
}
```

Java 16+: a record

```
record Range(int lo, int hi) {
    Range {
        if (lo > hi)
            throw new IllegalArgumentException(
                lo + " > " + hi);
    }
    int length() { return hi - lo; }
}
// new Range(1, 5).lo() → 1
// new Range(1, 5) →
//   Range[lo=1, hi=5]
// new Range(5, 1) → IAE
```

Right pane compiled and run on JDK 21. Note the accessor is `lo()`, not `getLo()`. Frameworks that look for JavaBean getters need record support, which Jackson, Spring and Hibernate all have today.

From that header, the compiler generates a **private final field** and a **public accessor per component**, a **canonical constructor**, and **equals**, **hashCode** and **toString**. (Oracle, Record Classes) `javap` on our `Range` shows the rest of the contract: `final class Range extends java.lang.Record`. You can't extend a record, and a record can't extend anything else. It can implement interfaces.

You can add **static fields**, **static factory methods**, **instance methods** and **nested types**. You can't add **instance fields** or instance initializers. (Oracle) Try `record Bad(int x) { int y; }` and `javac` answers: `field declaration must be static`. That's the transparency rule at work: if state isn't in the header, it doesn't exist.

2 · THE COMPACT CONSTRUCTOR: WHERE INVARIANTS LIVE

The compact constructor is the one feature that turns a record from a tuple into a type. You write it without a parameter list. The parameters are implicit, and **the fields are assigned automatically at the end**. (Oracle) Because every way of creating a record goes through it, it can do three jobs:

- **Validate.** Throw on bad input, as `Range` does above.
- **Normalize.** Reassign a parameter before the implicit assignment: `currency = currency.toUpperCase(Locale.ROOT);` stores `"EUR"` even if you passed `"eur"`. (We ran it.)
- **Defensively copy.** This one matters most, and it's where most people slip.

A record holding a list: naive vs correct

Naive: shallow immutability

```
record Team(String name,
            List<String> members) {}

var src = new ArrayList<>(
    List.of("ana", "bo"));
var t = new Team("core", src);
src.add("eve");
t.members().size();    // 3 (!)
t.members().add("x");  // works (!)
```

Correct: copy in the constructor

```
record Team(String name,
            List<String> members) {
    Team {
        members = List.copyOf(members);
    }
}

var t = new Team("core", src);
src.add("eve");
t.members().size();    // 2
t.members().add("x");  // throws UOE
```

Both compiled and run on JDK 21. `List.copyOf` (Java 10+) returns an unmodifiable list, rejects nulls, and generally doesn't copy a list that's already unmodifiable.

A record's fields are final, but `final` only freezes the reference. If a component is a mutable object, callers can still change what it points to, and they can change your `equals` and `hashCode` with it. Copy mutable inputs on the way in, and the accessor can return the field as is.

3 · WHERE RECORDS FIT, AND WHERE THEY DON'T

✓ Great fit

- DTOs and API request/response bodies
- Query projections (JPQL `select new ...`)
- Map keys, composite IDs, value objects (Money, Range)
- Config properties (Spring Boot binds to records)
- Multiple return values from a private method

VS

✗ Poor fit

- JPA entities: need a no-arg constructor, non-final class and fields
- Anything with identity or a lifecycle that mutates
- Types that need a superclass
- Public APIs whose internal representation may change

The JPA case is the one you'll meet first. An entity needs a no-arg constructor, must not be `final` (so the provider can create lazy-loading proxies) and needs non-final fields to map and dirty-check. Records break all three. (Thorben Janssen) Use them for the **read side** instead: projections, DTOs and embeddables.

The last row is the subtle one. A record's API is its representation: the components become the constructor, the accessors and the `toString`. Rename a component or split one in two, and every caller breaks. For a public library type whose internals may change, a normal final class with a private representation is still the better choice. For a DTO inside your service, that tight coupling is the whole point.

4 · THREE EVERYDAY PATTERNS

Static factories for meaningful names. The canonical constructor takes every component, in order. When some calls always pass the same value, add a static factory instead of a second constructor: `static Money eur(String v) { return new Money(new BigDecimal(v), "EUR"); }`. It still goes through the compact constructor, so validation can't be skipped.

"Withers" instead of setters. A record can't change, so "changing" one means building a new one. Write small `withX` methods that call the canonical constructor:

```
record Range(int lo, int hi) {
    Range { if (lo > hi) throw new IllegalArgumentException(); }
    Range withHi(int newHi) { return new Range(lo, newHi); }
}
// new Range(1, 5).withHi(9) → Range[lo=1, hi=9]
// new Range(1, 5).withHi(0) → IllegalArgumentException
```

Because every wither goes back through the constructor, **an invalid record can never exist**. That's the guarantee a setter-based POJO can't give you.

Local records for intermediate results. Instead of a `Map.Entry` or an `Object[]` in the middle of a stream, declare a record inside the method. `record Scored(String name, int score) {}` gives the intermediate value a name, real accessors and a readable `toString` for debugging, and it's invisible outside the method.

In the real world: Spring Boot and Hibernate are built for records now

Spring Boot binds `@ConfigurationProperties` straight into a record. Its reference docs say that a single parameterized constructor implies constructor binding, and that **"unless your record has multiple constructors, there is no need to use `@ConstructorBinding`"**. Defaults come from `@DefaultValue` on a component. One catch: the class must be enabled with `@EnableConfigurationProperties` or property scanning, not declared as a `@Component`. (Spring Boot docs)

Hibernate 6.2 (March 2023) added mapping **Java records as embeddables**, so an `@Embeddable` record `Name(String first, String last)` can sit inside an entity. (Hibernate blog) The entity itself stays a class, but its value parts can be records.

Gotcha: arrays in records

The generated `equals` compares each component with its own `equals`, and **arrays don't override `equals`**. We checked on JDK 21: `new Blob(new byte[]{1,2}).equals(new Blob(new byte[]{1,2}))` is **false**, while `Arrays.equals` on the two arrays is **true**. The `hashCode` is identity-based too, so two "identical" blobs are different `HashMap` keys. The array is also mutable through the accessor. **Prefer a List (copied with `List.copyOf`)**. If you must hold an array, clone it in the compact constructor and the accessor, and override `equals`, `hashCode` and `toString` with the `Arrays` helpers. At that point, ask whether a record still fits.

Under the hood

- **Serialization goes through the front door.** A normal `Serializable` class is rebuilt by reflection, without running its constructor. A record is **deserialized by calling its canonical constructor**, so your compact-constructor validation runs on untrusted streams too. You can't customize it with `readObject / writeObject`. (Oracle) (inside.java)
- **Records are reflectable as records.** `Range.class.getRecordComponents()` returns `[int lo, int hi]` in declaration order. That's how frameworks can learn component names and find the canonical constructor without relying on getters.
- **Nested and local records are implicitly `static`**, so they can't capture an enclosing instance or local variables. JEP 395 calls out this "no hidden state" rule. (JEP 395)
- **Annotations on components propagate** to the field, accessor and constructor parameter, wherever the annotation's `@Target` allows. That's why `@NotBlank String url` works for Bean Validation in a config record. (Oracle)

Try it (15 minutes, jshell on JDK 17+)

1. **Build a Money value.** `record Money(BigDecimal amount, String currency) ,` with a compact constructor that rejects null and negative amounts and upper-cases the currency. Check that `new Money(new BigDecimal("4.50"), "eur")` prints `Money[amount=4.50, currency=EUR]`.
2. **Break, then fix, immutability.** Write `record Team(String name, List<String> members) {}`, mutate the source list after construction, and watch `members()` change. Add `members = List.copyOf(members);` and try again.
3. **Write a "with"er.** Add `Range withHi(int hi) { return new Range(lo, hi); }`. Confirm that `withHi` still goes through your validation: `new Range(1, 5).withHi(0)` should throw.

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Act-now urgency

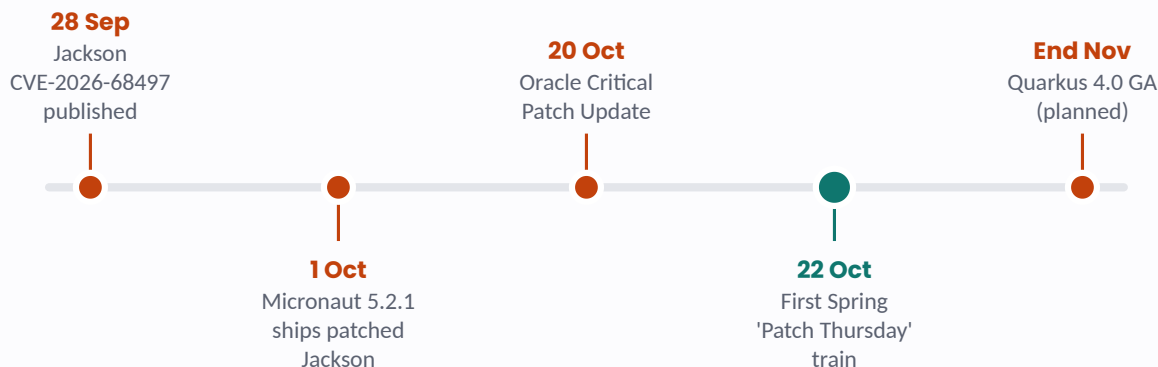


GO DEEPER

- **JEP 395: Records** (OpenJDK). The goals, non-goals and every restriction, with the reasoning.
- **Record Classes** (Oracle, Java SE 25 language guide). Compact constructors, local records, serialization.
- **Records tutorial** (dev.java). A guided walkthrough with examples.
- **Simpler serialization with records** (inside.java). Why constructor-based deserialization is safer.

~8 min • A quiet weekend for releases, so we look at two things that shape your patching in October: Spring's new monthly release train, and a Jackson CVE that touches both Jackson 2 and 3.

Your October patch calendar



Sources: GitLab advisory, Micronaut, Oracle security alerts, spring.io, Quarkus blog.

Top story 1 • Spring moves to one release day a month

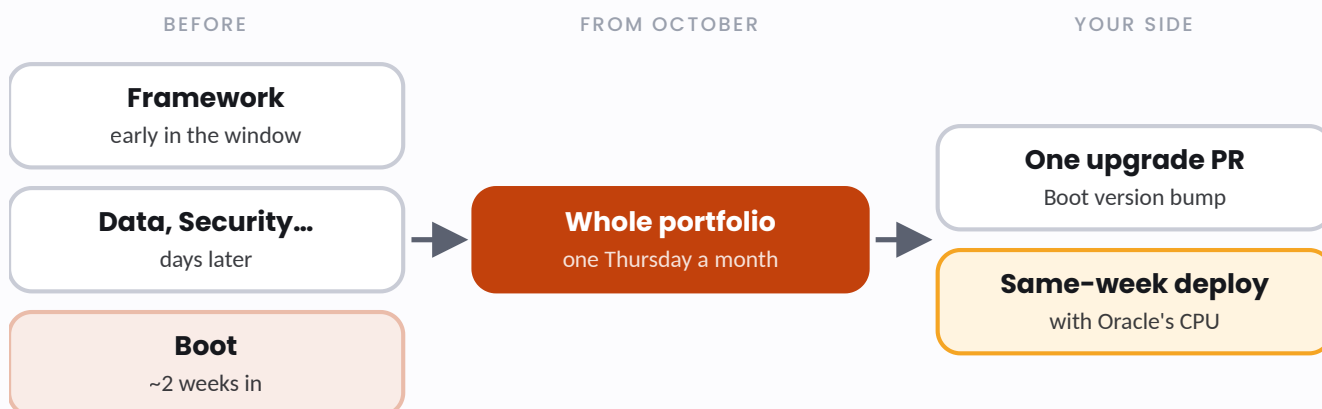
PLAN FOR IT

SECURITY

What happened. The Spring team has **re-engineered how it releases the whole portfolio**. Instead of releases spread over about two weeks, every Spring project now ships **"in one large train"** on a single day: **the Thursday after the third Monday of each month**, the same rhythm Spring Boot already used. The September milestones on 24 September were the dry run. **The first patch releases under the new model are due on 22 October.** (spring.io)

The reason is security. The team says it has averaged **nearly 80 new community-reported security reports a month since March**, with vulnerabilities now exploited within hours rather than days, and that it recently fixed **over 160 CVEs**. It has also rebuilt **spring.io/security** so you can filter advisories by CVE ID, severity and project. (spring.io)

Old vs new: how a Spring fix reaches you



Simplified from the spring.io announcement. Release day = the Thursday following the third Monday of each month.

Why it matters. Before, a fix in Spring Framework could land days before the Boot release that pulled it in, and you had to choose: override one version, or wait. **One train means one coordinated set of versions on a known day**, so a Boot bump picks up every fixed project at once. This month it also lands two days after Oracle's quarterly CPU on 20 October, so one patch week can cover both. (Oracle security alerts)

It also changes what a milestone month looks like. Between 23 and 25 September, Spring Boot 4.2.0-M2, Spring Data 2026.1.0-M2, Spring Security 7.2.0-M2, Spring Batch 6.1.0-M2, Spring Integration and AMQP milestones and the first Spring Cloud 2026.0 milestone all landed. (spring.io releases) **Expect the same clustering every month from now on:** one big day, then quiet.

The flip side: **80 reports a month means patch releases will keep carrying security fixes.** A team that bumps Spring Boot "when we get around to it" will drift further behind each month.

 **What to do.** Put a **standing monthly slot** in the calendar for the Friday after release day (23 October first), and let Dependabot or Renovate open the Boot bump PR automatically. Bookmark spring.io/security and filter it by the projects you actually use. On Boot 4.0, also note that its open-source support ends on **31 December 2026** (endoflife.date), so a 4.1 move belongs in this quarter's plan.

WHY IT MATTERS TO YOU

 Day-to-day impact

 Act-now urgency

 Interview value

Top story 2 · Jackson CVE-2026-68497: small request, big CPU bill

PATCHJSON

What happened. A denial-of-service flaw in **jackson-databind** was published on 28 September with a **CVSS score of 7.5 (High)**. When Jackson deserializes a JSON string into a `javax.xml.datatype.Duration` or `XMLGregorianCalendar` field, it passes the raw string to the JDK's `DatatypeFactory` with no length limit. An attacker can send a small request with an extremely long number and trigger expensive `BigInteger` parsing. No authentication is needed. (GitLab advisory) (CVE record via Vulners)

Line	Affected	Fixed in
Jackson 2 (<code>com.fasterxml</code>)	2.0.0 → 2.18.9, 2.19.0 → 2.21.5, 2.22.0 → 2.22.1	2.18.10, 2.21.6, 2.22.2
Jackson 3 (<code>tools.jackson</code>)	3.0.0 → 3.1.5, 3.2.0 → 3.2.1	3.1.6, 3.2.2

Sources: **CVE record**, **GitLab advisory**.

The ecosystem is already moving. **jOOQ** opened an upgrade issue on 1 October (jOOQ #20265), and **Micronaut 5.2.1** (1 October) moves to Jackson 3.2.3 "to address a vulnerability" and recommends that users update. (Micronaut)

Why it matters. The vulnerable path needs a `Duration` or `XMLGregorianCalendar` somewhere in a type you deserialize from untrusted input. **That's rare in new code, but common in SOAP-to-REST bridges and JAXB-generated classes**, where `XMLGregorianCalendar` is the default date type. It's the same family of problem as the 2011 hash-flooding attacks from yesterday's issue: **cheap for the attacker, expensive for the server**. And

since almost every Spring, Quarkus and Micronaut app has jackson-databind on its classpath, the version check is worth doing even if you think you don't use those types.

 **What to do.** Run `mvn dependency:tree -Dincludes=*:jackson-databind` (or `gradle dependencyInsight --dependency jackson-databind`) to see which version you actually resolve. If it's in an affected range, override it through your framework's managed property (in Spring Boot, the Jackson BOM version property) or wait for your framework's next patch, Spring's on 22 October. Meanwhile, search your DTOs for `XMLGregorianCalendar` and `javax.xml.datatype.Duration`. On public endpoints, replace them with `java.time` types.

Jargon buster

- **CVSS 7.5, AV:N/AC:L/PR:N/UI:N** : reachable over the network, low complexity, no privileges, no user interaction. The impact is availability only (**A:H**) : nothing leaks, the server just stops answering. (GitLab advisory)
- **BOM (bill of materials)**: a Maven POM that pins versions for a family of artifacts. Spring Boot's dependency management works this way, which is why you override one property instead of editing every Jackson artifact.
- **Release train**: many projects shipped together as one tested set of versions. Spring Cloud has long used named trains; now the whole Spring portfolio ships like one.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



Quick hits



Micronaut 5.2: Python on the JVM.

Micronaut 5.2.0 (27 September) adds **experimental Micronaut for Python**, built on GraalPy, plus a new Jakarta EL 6.0 module that compiles expressions at build time, the HTTP `QUERY` method and lower cold-start overhead. (Micronaut)



Micronaut 5.2.1.

Besides the Jackson bump, it requires lz4-java 1.11.2 in the Kafka module to resolve another vulnerability, and fixes request-body leaks in filters and HTTP/2 client window updates. (Micronaut)



Spring AI 2.1.0-M1.

Built on Spring Boot 4.2, it models messages as an **ordered list of parts** (text, reasoning, tool calls, media), adds an `OpenAiResponsesChatModel` for OpenAI's Responses API, and lets `VectorStore.upsert()` take pre-computed embeddings. (spring.io)



Spring Cloud 2026.0.0-M1 "Paddington".

The first milestone of the next release train, based on Boot 4.2.0-M2. It brings Resilience4j 2.4.0, OpenFeign on Feign 13.14, and Gateway retries moved from Reactor Addons to Spring Framework core. (spring.io)

Spring Boot support clock.

Open-source support for **Boot 4.0 ends 31 Dec 2026** and for **4.1 on 31 Jul 2027**. Boot 3.5's open-source support already ended on 30 June 2026 (commercial support runs much longer). ([endoflife.date](#))

Coming up.

Oracle's CPU on **20 October**, Spring's first monthly patch train on **22 October**, Spring Boot 4.2 RC1, Quarkus 4.0 GA at the end of November, and JDK 28 in March 2027.

Revision card

- **A record = final class extending `java.lang.Record`**, with private final fields, accessors (`x()`, not `getX()`), a canonical constructor, `equals`, `hashCode` and `toString`.
- **No instance fields, no superclass.** Static members, instance methods and interfaces are fine.
- **Compact constructor:** validate, normalize, copy. Fields are assigned automatically at the end.
- **Immutability is shallow.** Use `List.copyOf` for collections; avoid arrays (their `equals` is identity).
- **Not for JPA entities.** Use records for DTOs, projections, keys, config and (since Hibernate 6.2) embeddables.
- **Records deserialize through the canonical constructor**, so your validation also guards untrusted streams.
- **Spring ships monthly on one Thursday**, starting with patches on 22 Oct. Plan a standing upgrade slot.
- **Jackson CVE-2026-68497:** upgrade to 2.18.10 / 2.21.6 / 2.22.2 or 3.1.6 / 3.2.2.

Java – Beautifully Broken is researched and written by Claude every morning from OpenJDK, Oracle and Spring announcements, official docs, InfoQ and engineering blogs. Vendor numbers are labelled as claims, and every item links to its source. Want a topic sooner or a different depth? Change `java/settings.yml` or just reply. ☕